❖ Rest Framework

(1) Introduction to APIs

Q-1)What is an API (Application Programming Interface)?

- An **Application Programming Interface (API)** is a set of rules and protocols that allow different software applications to communicate and exchange data with one another. It acts as an intermediary or "messenger," enabling one system to request services or information from another without needing to understand its internal code.

- **Core Concepts & Functionality**

- APIs operate on a **Request-Response cycle** involving three main parties:

- **Client:** The application making the request (e.g., a mobile app).
- **Server:** The system that processes the request and holds the data.
- **The API:** The "waiter" that takes your order (request), tells the kitchen (server) what to do, and brings the food (response) back to you.

Q-2)Types of APIs: REST, SOAP.

- REST (Representational State Transfer) and SOAP (Simple Object Access Protocol) are the two primary methods for web service communication, each with distinct design philosophies and technical requirements.

• REST (Representational State Transfer)

- REST is an architectural style that focuses on resources (data) rather than functions. It has become the industry standard for modern web and mobile applications due to its flexibility and performance.

- **Data Formats:** Supports multiple formats including JSON (most common), XML, HTML, and plain text.
- **Protocol:** Operates primarily over HTTP/HTTPS and uses standard methods like GET, POST, PUT, and DELETE.
- **State:** Stateless; every request must contain all information necessary to be processed by the server.
- **Caching:** Inherently cacheable, which improves speed and reduces server load.
- **Best For:** Public APIs, mobile apps, social media, and microservices.

• SOAP (Simple Object Access Protocol)

- SOAP is a standard messaging protocol designed for strict, secure, and reliable communication, often used in complex enterprise environments.

- **Data Formats:** Exclusively uses XML for its message structure.
- **Protocol:** Protocol-agnostic; it can work over HTTP, SMTP, TCP, JMS, and more.
- **State:** Can be either stateless or stateful, supporting complex, multi-step transactions.
- **Security:** Features high-level, built-in security standards like WS-Security, which allows for message-level encryption and digital signatures.
- **Reliability:** Includes built-in error handling (Fault elements) and ACID compliance to ensure reliable database transactions.
- **Best For:** Financial services (banking), healthcare systems, and legacy enterprise integrations.

- ## Key Comparison Summary

Feature	REST	SOAP
Type	Architectural Style	Protocol
Design	Resource-based (Nouns)	Action-based (Verbs)
Message Size	Lightweight (JSON)	Verbose (XML)
Contract	Optional (e.g., OpenAPI)	Required (WSDL)
Scalability	High (stateless)	Challenging (can be stateful)
Performance	Faster (less overhead)	Slower (high XML overhead)

Q-3)Types of APIs: REST, SOAP.

APIs (Application Programming Interfaces) are critical in web development because they serve as the "connective tissue" that allows different software systems to communicate and share data. They have transformed web development from building isolated websites to creating interconnected digital ecosystems.

- ## Key Reasons for the Importance of APIs

- **Code Reusability and Efficiency:** APIs allow developers to integrate pre-built functionalities (like Stripe for payments or Google Maps for location services) instead of building them from scratch. This significantly accelerates the software development lifecycle.
- **Separation of Concerns:** APIs act as a contract between the frontend and backend, enabling teams to work independently. Changes to the user interface (frontend) won't break the database logic (backend) as long as the API remains consistent.
- **Scalability:** They support microservices architecture, where applications are broken into smaller, independent services. This allows developers to scale specific parts of an application—such as payment processing—independently based on demand.
- **Interoperability and Multi-Platform Support:** A single API can power multiple platforms, including web browsers, mobile apps (iOS/Android), and IoT devices. This ensures data consistency and a unified experience across all user touchpoints.
- **Enhanced Security:** APIs provide a controlled gateway for accessing sensitive data. They use authentication tokens (like OAuth) and API keys to verify users and applications, keeping internal system details hidden from external threats.
- **Automation and Innovation:** APIs drive automation by allowing machines to talk to each other without human intervention. They also foster innovation by enabling developers to build on top of existing platforms (e.g., using the OpenAI API to add AI features to a custom website).

(2) Requirements for Web Development Projects

Q-1) Understanding project requirements

- The Django REST Framework (DRF) is a powerful, flexible toolkit for building web APIs on top of Django, ideal for creating scalable backend services for web/mobile apps. Key requirements include proficiency in Python, Django basics, RESTful principles, and data serialization (JSON). It enables robust authentication, serialization, and viewset handling to deliver efficient APIs.

- **Key Project Requirements for DRF Development:**

- **Core Dependencies**: Requires Python (3.10+) and Django (4.2+)

- **Database Management:** Utilize Django models and ORM for data retrieval and storage.

- **Serialization:** Use DRF Serializers to transform complex data types (e.g., ORM models) into JSON or other content types.

- **API Views:** Implement DRF Views (Function-based or Class - based) to manage API requests and responses.

- **Routing:** Use DRF Routers for automatic URL configuration.

- **Authentication & Permissions**: Set up OAuth1a, OAuth2, or token authentication, along with object-level permissions, as explained in this Medium article.

- **Documentation:** Utilize the built-in, browsable API feature for testing and documentation.

- ## When to Use DRF

➢ According to iFlair Web Technologies, DRF is best for applications needing secure, modern backends, such as:

- Single Page Applications (SPAs) (React, Vue, Angular)
- Mobile application backends
- Microservices architectures
- Complex e-commerce platforms.

Q-2) Setting up the environment and installing necessary packages.

➢ To set up a Django REST Framework (DRF) environment, follow these sequential steps to ensure all dependencies are isolated and correctly configured.

1. Initialize the Environment

➢ It is recommended to use a virtual environment to manage dependencies independently of your operating system.

- **Create a project directory**:

```bash
mkdir my_api_project
cd my_api_project
```

- **Create and activate a virtual environment**:
  - **macOS/Linux**: python3 -m venv .venv followed by source .venv/bin/activate.
  - **Windows**: python -m venv .venv followed by .venv\Scripts\activate.

2. Install Necessary Packages

➢ Use the Python Package Index (PyPI) via pip to install the core frameworks and optional supports.

- **Core Packages**:

  pip install django djangorestframework

- **Recommended Optional Packages**:
  - **Markdown**: pip install markdown (Adds support for the browsable API).
  - **Filtering**: pip install django-filter (Enables API filtering capabilities).
  - **Syntax Highlighting**: pip install pygments (For code highlighting in the browsable API).

3. Create and Configure the Project

- **Start a Django project and app**:

  django-admin startproject project_name .

  python manage.py startapp api_app

- **Register DRF in settings.py**:

➢ Add 'rest_framework' and your new app to the INSTALLED_APPS list.

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'api_app',
]
```

4. Finalize Setup

- **Run migrations**: This sets up the default database tables.

```
python manage.py migrate
```

- **Enable Authentication for the Browsable API**: Add this to your root urls.py to allow logging in/out directly from the browser.

```
urlpatterns = [
    ...
    path('api-auth/', include('rest_framework.urls')),
]
```

- **Start the server**: Use the Django development server to verify the installation.

```
python manage.py runserver
```

(3) Serialization in Django REST Framework

Q-1) What is Serialization?

- serialization is the process of converting complex data structures—such as Django model instances or querysets—into native Python datatypes that can then be easily rendered into formats like JSON or XML. It also handles the reverse process, known as deserialization, which converts incoming data back into complex types after validating it.

- **Key Functions of Serializers**

- **Data Conversion**: They bridge the gap between your database (Django models) and the API client by transforming objects into a transmittable format (e.g., JSON).
- **Validation**: Serializers ensure that incoming data meets specific criteria before it is processed or saved to the database.
- **CRUD Support**: They provide methods like .create() and .update() to simplify saving and modifying database records based on validated data.

Q-2) Converting Django QuerySets to JSON.

- In Django, a QuerySet cannot be directly converted to JSON because it is not a serializable Python object. You must first transform it into a compatible format (like a list of dictionaries) or use a dedicated serializer.

- **Common Methods to Convert a QuerySet to JSON**

  - **Using Django's Built-in Serializers**

    This is the most direct way to get a JSON-formatted string from a QuerySet. It includes model metadata like the primary key and model name by default.

```
from django.core import serializers
from django.http import HttpResponse

def my_view(request):
    queryset = MyModel.objects.all()
    # Serializes directly to a JSON string
    data = serializers.serialize("json", queryset)
    return HttpResponse(data,
content_type="application/json")
```

- **Using JsonResponse with .values()**

➢ This method is preferred for lightweight responses where you only need a simple list of dictionaries without extra model metadata.

```
from django.http import JsonResponse

def my_view(request):
    # .values() returns a ValuesQuerySet (list-like)
    data = list(MyModel.objects.values('id', 'name', 'created_at'))
    # safe=False is required for top-level lists
    return JsonResponse(data, safe=False)
```

- **Using Django REST Framework (DRF) Serializers**

➢ For professional APIs, Django REST Framework is the industry standard. It handles nested relationships, validation, and complex data types automatically.

```
from rest_framework import serializers
from rest_framework.response import Response
from rest_framework.decorators import api_view

class MyModelSerializer(serializers.ModelSerializer):
    class Meta:
        model = MyModel
        fields = '__all__'
```

```
@api_view(['GET'])
def my_view(request):
    queryset = MyModel.objects.all()
    # many=True is mandatory for serializing a QuerySet
    serializer = MyModelSerializer(queryset, many=True)
    return Response(serializer.data)
```

Q-3) Using serializers in Django REST Framework (DRF).

➢ In Django REST Framework (DRF), serializers act as the bridge between complex data types—like Django model instances or querysets—and native Python datatypes that can be rendered into formats like JSON or XML. They also handle deserialization, which converts incoming data back into complex types after validating it.

- **Core Types of Serializers**

- **Serializer**: The base class for manual field definitions. It provides maximum control and can be used for non-model data.

- **ModelSerializer**: A shortcut class that automatically generates fields based on a Django model, including default validators and .create()/.update() methods.

- **HyperlinkedModelSerializer**: Similar to ModelSerializer but uses hyperlinks (URLs) to represent relationships instead of primary keys.

- ## Key Functionalities

1. Serialization (Read): Converting a model instance into a dictionary using serializer.data. To serialize multiple objects, pass many=True to the serializer constructor.

2. Deserialization & Validation (Write): Converting request data into validated Python objects. You must call .is_valid() before accessing .validated_data or calling .save().

3. Saving Data: Calling .save() will either create a new instance or update an existing one, depending on whether an instance was passed during initialization.

4. Custom Validation:

1. Field-level: Implement a method named validate_<field_name>(self, value).

2. Object-level: Override the validate(self, data) method to handle multi-field validation.

- ## Advanced Features

- **Nested Serializers**: Used to represent related models (e.g., a User inside a Comment). This is achieved by declaring a serializer as a field in another.

- **SerializerMethodField**: A read-only field that gets its value by calling a method on the serializer class, typically named get_<field_name>.

- **Depth Argument**: In ModelSerializer, setting depth = 1 (or higher) automatically expands nested relationships into their full representations rather than just IDs.

- **The source Argument**: Allows you to map a field to a different model attribute, call a model method, or traverse relationships (e.g., source='user.profile.bio').

- **Context**: You can pass extra information to a serializer using the context argument, accessible within the serializer as self.context.

(4) Requests and Responses in Django REST Framework

Q-1) HTTP request methods (GET, POST, PUT, DELETE).

- In Django REST Framework (DRF), HTTP methods are mapped to specific actions to perform CRUD (Create, Read, Update, Delete) operations on resources.

  - **Core HTTP Request Methods**

    - DRF provides built-in support for these methods through the APIView class or @api_view decorator:

- **GET**: Used to retrieve data from the server.
  - GET /resource/: Lists all instances (Collection).
  - GET /resource/<id>/: Retrieves a single instance (Individual).
  - Characteristics: Safe (does not modify data) and idempotent.
- **POST**: Used to create a new resource.
  - Data is sent in the request body.
  - Characteristics: Neither safe nor idempotent (repeated requests may create duplicate entries).
- **PUT**: Used to update or replace an existing resource entirely.
  - Requires a complete representation of the resource in the request body.
  - Characteristics: Idempotent (multiple identical requests have the same effect as one).

- **PATCH**: Used for partial updates to a resource.
  - Only the fields being changed need to be sent in the request body.
- **DELETE**: Used to remove a specific resource.
  - Typically targets a specific ID, e.g., DELETE /resource/<id>/.
  - Characteristics: Idempotent (deleting something that is already gone results in the same final state).

Q-2) Sending and receiving responses in DRF

➢ In Django REST Framework (DRF), the interaction between a client and the server is centered around a specialized Request-Response Cycle that extends standard Django functionality to better handle web APIs.

1. Receiving Requests

➢ When a client sends data, DRF wraps the standard Django HttpRequest into a more powerful Request object.

- **request.data**: This is the primary way to access incoming data. Unlike request.POST, it handles multiple formats (like JSON or XML) and works for POST, PUT, and PATCH methods.
- **Content Negotiation**: DRF automatically determines how to parse the request body based on the Content-Type header sent by the client.
- **Validation**: Incoming data is typically passed to a Serializer which validates the data using .is_valid() before any processing or saving to the database occurs.

2. Sending Responses

- Instead of returning a standard HttpResponse, DRF views return a Response object which is more flexible for API development.

- **Response(data)**: You initialize a response with native Python primitives (like a dictionary). DRF then uses Renderers to convert that data into the format requested by the client (usually JSON).
- **Explicit Status Codes**: DRF provides human-readable constants for HTTP status codes, such as status.HTTP_201_CREATED or status.HTTP_400_BAD_REQUEST, ensuring your code is clear and accurate.
- **Customizing Output**: You can modify the final structure of a response by overriding the to_representation method in your serializer or by customizing the Response Class directly.

(5) Views in Django REST Framework

Q-1) Understanding views in DRF: Function-based views vs Class-based views.

- In Django REST Framework (DRF), views are responsible for handling HTTP requests and returning responses. They can be implemented as simple functions or as classes, each offering different trade-offs in terms of simplicity and reusability.

1. Function-Based Views (FBVs)

- Function-based views are simple Python functions that receive a request and return a response.

- **How they work**: In DRF, you must use the @api_view decorator to wrap a function, which ensures it receives a REST framework Request instance and can return a REST framework Response.

- **Best for**: Small, straightforward logic, quick prototyping, or one-off views that don't fit standard patterns.

- **Pros**: Easy to read, lower learning curve for beginners, and explicit control over every line of logic.

- **Cons**: Can lead to code duplication and long, messy functions with many if/elif blocks to handle different HTTP methods (e.g., if request.method == 'GET').

2. Class-Based Views (CBVs)

- Class-based views use object-oriented principles to organize code into separate methods for each HTTP verb.

- **How they work**: You subclass APIView and define methods like .get(), .post(), and .delete(). DRF's dispatch() method automatically routes the request to the correct method.

- **Best for**: Complex views, large projects, and scenarios where you want to reuse code through inheritance.

- **Pros**: Promotes DRY (Don't Repeat Yourself) code, uses inheritance and mixins for reusability, and offers a cleaner structure for multiple HTTP methods.

- **Cons**: Steeper learning curve due to high abstraction; it can be harder to "see" the logic buried in parent classes.

(6) URL Routing in Django REST Framework

Q-1) Defining URLs and linking them to views.

- In Django REST Framework (DRF), URL routing maps incoming web requests to specific view logic. You can define these routes manually using standard Django path functions or automatically using DRF's built-in Routers.

1. Manual Routing with path()

- For basic function-based views or class-based views (APIView), you manually define each endpoint in urls.py.
  - **Function-Based Views**: Use the @api_view decorator and link it directly.

```
from django.urls import path
from .views import my_function_view

urlpatterns = [
    path('my-endpoint/', my_function_view, name='my-view'),
]
```

- **Class-Based Views**: Call .as_view() to link the class to a route.

```
from .views import MyAPIView

urlpatterns = [
    path('api/resource/', MyAPIView.as_view(), name='resource-list'),
]
```

2. Automatic Routing with Routers

➢ When using ViewSets, DRF's Routers automatically generate standard CRUD URL patterns for you
(e.g., list, create, retrieve, update, destroy).

- **DefaultRouter**: This is the most common router. It automatically creates an API root view with links to all registered resources.
- **SimpleRouter**: Similar to DefaultRouter but does not include a default API root view.
- **Implementation Steps**:
    1. Initialize the router.
    2. Register your ViewSet with a URL prefix and a basename.
    3. Include the router's URLs in your urlpatterns list.

```
from rest_framework.routers import DefaultRouter
from .views import MyViewSet

router = DefaultRouter()
router.register(r'my-resource', MyViewSet, basename='myresource')
```

```
urlpatterns = [
    path('', include(router.urls)),
]
```

3. Routing Custom Actions

- If you add custom methods to a ViewSet using the @action decorator, the router automatically generates a URL for them.

- **Detail actions**: (e.g., /users/1/set-password/) are created when detail=True is set.
- **List actions**: (e.g., /users/recent-users/) are created when detail=False is set.

4. Capturing URL Parameters

- You can capture dynamic data (like IDs or slugs) using Path Converters in the URL string:

  - <int:pk>: Matches an integer ID.
  - <str:username>: Matches any string.
  - <slug:post_title>: Matches letters, numbers, and hyphens.

(7) Pagination in Django REST Framework

Q-1) Adding pagination to APIs to handle large data sets.

- Django REST Framework (DRF) provides built-in support for splitting large datasets into manageable parts. Pagination is automatically applied when using Generic Views or ViewSets if a pagination class is configured.

1. Built-in Pagination Styles

- **PageNumberPagination**: The most common style, where results are requested by page number (e.g., ?page=2). It includes metadata like total count and next/previous links.
- **LimitOffsetPagination**: Mirroring SQL syntax, it uses limit (number of items) and offset (starting position) parameters (e.g., ?limit=10&offset=20).
- **CursorPagination**: An opaque, cursor-based style that is highly efficient for extremely large or rapidly changing datasets. It prevents duplicate results during concurrent updates but does not allow jumping to arbitrary pages.

2. Configuration Options

- **Global Setup**

- To apply pagination to all views in your project, add these settings to your Django settings.py:

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 100
}
```

Note: Both a class and a page size must be defined for global pagination to activate.

- **Per-View Setup**

➢ To use a specific style or different page size for a single view, set the pagination_class attribute:

```
from rest_framework.pagination import PageNumberPagination

class MyListView(generics.ListAPIView):
    queryset = MyModel.objects.all()
    serializer_class = MySerializer
    pagination_class = PageNumberPagination
```

To disable pagination for a specific view, set pagination_class = None.

3. Custom Pagination

➢ You can extend built-in classes to modify behavior, such as changing query parameter names or adding custom keys to the response:

```
from rest_framework.pagination import PageNumberPagination
from rest_framework.response import Response

class StandardResultsSetPagination(PageNumberPagination):
    page_size = 100
    page_size_query_param = 'page_size'  # Allows client to set custom page size
    max_page_size = 1000

    def get_paginated_response(self, data):
        return Response({
            'links': {
                'next': self.get_next_link(),
                'previous': self.get_previous_link()
            },
            'count': self.page.paginator.count,
            'results': data
        })
```

(8) Settings Configuration in Django

Q-1) Configuring Django settings for database, static files, and API keys.

- To configure a Django project, settings are primarily managed in the settings.py file, utilizing environment variables for security.

1. Database Configuration

- Django supports multiple backends. While it defaults to SQLite for local development, production typically uses PostgreSQL.

- **SQLite (Local)**:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

- **PostgreSQL (Production)**:

- Requires installing psycopg2-binary.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'your_db_name',
        'USER': 'your_db_user',
        'PASSWORD': 'your_db_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

2. Static Files Configuration

- Static files (CSS, JS, images) require specific paths to be recognized during development and collected for production.

- **STATIC_URL**: The URL prefix used to access static files in the browser (e.g., '/static/').
- **STATICFILES_DIRS**: A list of additional directories where Django looks for static files during development (e.g., [BASE_DIR / 'static']).
- **STATIC_ROOT**: The absolute path to the directory where all static files are collected for production using python manage.py collectstatic.

3. API Keys and Environment Variables

- Never hardcode sensitive information like API keys or the SECRET_KEY in settings.py. Instead, use an .env file and a library like python-decouple or python-dotenv.

1. Create an .env file in the project root:

```
SECRET_KEY=your-secret-key
STRIPE_API_KEY=your-stripe-key
```

2. Load in settings.py:

```
from decouple import config
SECRET_KEY = config('SECRET_KEY')
STRIPE_API_KEY = config('STRIPE_API_KEY')
```

3. **Security**: Always add .env to your .gitignore to prevent it from being committed to version control.

(9) Project Setup

Q.1) Setting up a Django REST Framework project.

- Setting up a Django REST Framework (DRF) project involves installing the necessary packages, configuring your Django settings, and building the core components: models, serializers, and views.

1. Environment and Package Setup

- First, isolate your project using a virtual environment and install Django along with DRF.

- **Create Environment**: python -m venv .venv.
- **Activate**: source .venv/bin/activate (Linux/macOS) or .venv\Scripts\activate (Windows).
- **Install Packages**: pip install django djangorestframework.

2. Project Initialization

- Create a new Django project and a dedicated app for your API.
- Start Project: django-admin startproject myproject ..
- Start App: python manage.py startapp api.

- Register: In settings.py, add 'rest_framework' and your app name to INSTALLED_APPS.

3. Core Development Workflow

➢ Follow these standard steps to build your first endpoint:

- **Define Models**: Create your data structure in api/models.py.
- **Apply Migrations**: Run python manage.py makemigrations followed by python manage.py migrate to sync the database.
- **Create Serializers**: In api/serializers.py, define classes that convert model instances into JSON.
- **Build Views**: Use DRF Generic Views or ViewSets in api/views.py to handle logic.
- **Configure URLs**: Map your views to URL paths in api/urls.py and include them in the project's root urls.py.

4. Verification and Testing

- **Run Server**: Start the development server with python manage.py runserver.
- **Browsable API**: Visit http://127.0.0.1:8000/api/ in your browser to interact with the DRF Browsable API interface.
- **Create Superuser**: Use python manage.py createsuperuser to access the admin panel and manage your API data.

(10)Social Authentication, Email, and OTP Sending API

Q.1) Implementing social authentication (e.g., Google, Facebook) in Django.

- Implementing social authentication in Django is most efficiently achieved using the django-allauth package. This tool abstracts the complexities of the OAuth2 protocol and supports over 50 providers.

1. Core Configuration

- To begin, install the library and integrate it into your Django settings:

- **Installation**: Run pip install django-allauth in your terminal.
- **InstalledApps**: Add django.contrib.sites, allauth, allauth.account, allauth.socialaccount, and the specific provider apps (e.g., allauth.socialaccount.providers.google) to INSTALLED_APPS.
- **AuthenticationBackends**: Include allauth.account.auth_backends.AuthenticationBackend in the AUTHENTICATION_BACKENDS list to allow social logins alongside standard username/password methods.
- **Site ID**: Set SITE_ID = 1 in settings.py (required by the Django sites framework).

2. Provider-Specific Setup

- Each social provider requires you to register your application on its developer platform to obtain a Client ID and Client Secret.

- **Google**: Use the Google Cloud Console to create a new project and configure an "OAuth client ID." Ensure the "Authorized redirect URIs" include http://127.0.0 for local testing.
- **Facebook**: Register your app at Meta for Developers. Add "Facebook Login" as a product and set the redirect URI to http://localhost:8000/accounts/facebook/login/callback/.
- **Django Admin Setup**: Log in to the Django Admin interface, navigate to Social Applications, and add a new entry for each provider with the credentials obtained above.

3. Integrating with Frontend

- After configuration, you must expose the authentication routes to your users.

- **URL Routing**: In your main urls.py, include the allauth patterns: path('accounts/', include('allauth.urls')).
- **Login Links**: Add dynamic links in your templates to trigger the login flow:
- **Google**: {% url 'socialaccount_login' 'google' %}
- **Facebook**: {% url 'socialaccount_login' 'facebook' %}
- **Redirects**: Define LOGIN_REDIRECT_URL = '/' to specify where users should go after a successful login.

4. Security & Production Best Practices

- **Environment Variables**: Store sensitive keys like the Client Secret in a .env file rather than hardcoding them in settings.py.
- **HTTPS**: For production, ensure all callback URLs use HTTPS, as many providers (like Facebook) strictly require it for secure token exchange.
- **Scopes**: Request only the minimal necessary data (e.g., ['email', 'profile']) to follow the principle of least privilege.

Q.2) Sending emails and OTPs using third-party APIs like Twilio, SendGrid.

➢ Integrating third-party APIs like SendGrid for emails and Twilio for SMS/OTPs into a Django project involves configuring backend settings and using their respective Python SDKs.

1. Sending Emails with SendGrid

➢ Twilio SendGrid offers an SMTP interface that integrates seamlessly with Django's native send_mail function.

- **Configuration**: Add these settings to your settings.py file:

```
EMAIL_HOST = 'smtp.sendgrid.net'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = 'apikey'  # This is the exact string 'apikey' for all accounts
```

```
EMAIL_HOST_PASSWORD = 'YOUR_SENDGRID_API_KEY'
DEFAULT_FROM_EMAIL = 'your-verified-email@example.com'
```

- **Sending Email**: Use Django's built-in send_mail utility in your views.py:

```
from django.core.mail import send_mail

send_mail(
    'Subject Here',
    'Here is the message content.',
    'from@example.com',
    ['recipient@example.com'],
    fail_silently=False,
)
```

- **Alternative API Approach**: For advanced features like dynamic templates, you can use the sendgrid-python library directly instead of SMTP.

2. Sending OTPs via SMS with Twilio

➢ Twilio's SMS API is typically used for phone-based OTP verification.

- **Setup**: Install the Twilio Python helper library: pip install twilio.
- **Credentials**: Store your ACCOUNT_SID, AUTH_TOKEN, and TWILIO_PHONE_NUMBER in environment variables or settings.py.
- **Implementation**: Create a helper function to generate a random code and send it:

```
from twilio.rest import Client
import random

def send_otp(phone_number):
    # Generate a 6-digit OTP
    otp = str(random.randint(100000, 999999))

    client = Client('TWILIO_ACCOUNT_SID',
'TWILIO_AUTH_TOKEN')
    message = client.messages.create(
        body=f"Your verification code is {otp}",
        from_='YOUR_TWILIO_PHONE_NUMBER',
        to=phone_number
    )
return otp
```

- **Verification Logic**: Store the generated OTP in the user's session or a temporary database model, then compare it against the user's input in a verification view.

(11) RESTful API Design

Q.1) REST principles: statelessness, resource-based URLs, and using HTTP methods for CRUD operations.

- The core REST (Representational State Transfer) principles define how networked applications should communicate to ensure scalability and simplicity.

1. Statelessness

- Statelessness requires that every HTTP request contains all the information needed for the server to fulfill it.
  - **No Session Context**: The server does not store any client session data between requests.
  - **Self-Contained Requests**: Each request must include its own context, such as authentication tokens (JWT) or necessary query parameters.
  - **Scalability**: This allows any server in a cluster to handle any incoming request, facilitating easy horizontal scaling.

2. Resource-Based URLs

- In REST, everything is treated as a resource (e.g., a user, a product, or a collection) identified by a unique URI (Uniform Resource Identifier).
  - **Nouns over Verbs**: URLs should represent things (nouns), not actions (verbs). For example, use /users instead of /getUsers.

- **Predictable Structure**: URLs often follow a hierarchical pattern, such as /orders/123/items to identify specific items within a specific order.
- **Collections vs. Individuals**: Plural nouns typically represent collections (e.g., /books), while an appended ID identifies a specific resource (e.g., /books/9780131101630).

3. HTTP Methods for CRUD Operations

➢ RESTful APIs use standard HTTP verbs to perform CRUD (Create, Read, Update, Delete) operations on resources.

CRUD Operation	HTTP Method	Action Description
Create	POST	Creates a new resource in a collection.
Read	GET	Retrieves the representation of a specific resource or collection.
Update	PUT / PATCH	PUT replaces the entire resource; PATCH performs a partial update.
Delete	DELETE	Removes a specific resource from the server.

(12) CRUD API (Create, Read, Update, Delete)

Q.1) What is CRUD, and why is it fundamental to backend development?

- CRUD is an acronym for Create, Read, Update, and Delete, representing the four fundamental operations used to manage data within a system. In backend development, a CRUD API allows client applications (like a web browser or mobile app) to interact with a server's database to perform these essential actions on resources.

- **The Four CRUD Operations**

- In a RESTful API architecture, these operations typically map to standard HTTP methods:
  - **Create**: Adds a new record to the database (e.g., registering a new user). It is performed using the POST method.
  - **Read**: Retrieves existing data from the system (e.g., viewing a profile). It is performed using the GET method and does not modify the data.
  - **Update**: Modifies an existing record (e.g., changing a password). It is performed using the PUT (replaces the entire record) or PATCH (updates only specific fields) methods.
  - **Delete**: Removes a record from the database (e.g., closing an account). It is performed using the DELETE method.

- **Why CRUD is Fundamental to Backend Development**

➢ CRUD is the bedrock of backend engineering for several reasons:

- **Complete Data Lifecycle**: Together, these four actions cover the entire life cycle of data in an application—from its initial creation to its eventual removal.
- **Standardization**: It provides a predictable, uniform framework for developers to manage data across different platforms and technologies.
- **Foundation for Complexity**: Most backend systems begin with basic CRUD logic. Mastering it allows developers to build advanced features like authentication, complex business logic, and real-time updates.
- **Universal Application**: Nearly every modern software application—whether for e-commerce, social media, or banking—is built on these four core data operations.
- **Separation of Concerns**: CRUD APIs help separate the frontend (user interface) from the backend (data management), allowing them to be developed and scaled independently.

(13) Authentication and Authorization API

Q.1) Difference between authentication and authorization.

In a REST API, authentication and authorization are sequential security steps that confirm a user's identity and determine their allowed actions, respectively.

- **Key Differences**

Feature	Authentication (AuthN)	Authorization (AuthZ)
Purpose	Verifies who the user or application is.	Determines what the verified user is allowed to do.
Question	"Are you who you say you are?"	"Do you have permission to access this resource?"
Timing	Occurs first.	Occurs after successful authentication.
Methods	Passwords, API keys, biometrics, OTPs.	Role-Based Access Control (RBAC), OAuth scopes, ABAC.
Transmits	Identity info (often via ID tokens).	Permission info (often via access tokens).

Example	Logging in with a username and password.	Being allowed to edit your own profile but not others'.

Q.2) Implementing authentication using Django REST Framework's token-based system.

- Implementing token-based authentication in Django REST Framework (DRF) involves exchanging user credentials for a unique string (token) used to identify the user in subsequent requests.

1. <u>Configuration</u>

- To enable the built-in token system, update your settings.py file:
  - **<u>Add App</u>**: Include rest_framework.authtoken in your INSTALLED_APPS.
  - **<u>Set Default Classes</u>**: Configure REST_FRAMEWORK to use TokenAuthentication and IsAuthenticated by default.
  - **<u>Run Migrations</u>**: Execute python manage.py migrate to create the database table for tokens.

2. Token Generation

➢ Tokens can be generated in several ways:

- **Built-in View**: Add the obtain_auth_token view to your urls.py to allow users to exchange their username and password for a token.

- **Signals**: Use Django's post_save signal on the User model to automatically generate a token whenever a new user is created.

- **Admin Interface**: Manually create and manage tokens via the Django Admin.

3. Using the Token

➢ Once a client obtains a token, it must be included in the Authorization header of every HTTP request:

- **Format**: Authorization: Token <your_token_string>.

4. Protecting Views

➢ You can restrict access to specific views by applying DRF permissions:

- **Class-Based Views**: Set permission_classes = [IsAuthenticated].

- **Function-Based Views**: Use the @permission_classes([IsAuthenticated]) decorator.

(14) OpenWeatherMap API Integration

Q.1) Introduction to OpenWeatherMap API and how to retrieve weather data.

- The OpenWeatherMap API is a widely-used online service providing global weather data, including real-time conditions, forecasts, and historical archives. It is accessible via industry-standard formats like JSON and XML, making it easy to integrate into web and mobile applications.

1. Getting Started

- To retrieve data, you must first obtain an API Key (also called APPID), which acts as your unique identifier for authentication and rate-limiting.
  - **Register**: Create a free account at OpenWeatherMap.org.
  - **API Key**: After email confirmation, locate your key under the "API Keys" section of your account dashboard.
  - **Activation**: New keys may take up to 2 hours to activate before they can process requests.

2. Basic API Call Structure

- Most standard requests follow a simple HTTP GET structure. For current weather, use the following URL format: https://openweathermap.org{city_name}&appid={your_API_key}&units={metric|imperial}

- **Parameters**:
  - q: The target city name (e.g., London).
  - appid: Your unique API Key.
  - units: Optional. Default is Kelvin. Use metric for Celsius or imperial for Fahrenheit.
  - lang: Optional. Change the response language (e.g., fr for French).

3. Key Data Products

- **Current Weather Data**: Real-time atmospheric conditions for any global location.
- **One Call API 3.0**: A single request that returns current weather, minute-by-minute forecasts, hourly (up to 48 hours), and daily (up to 8 days) data.
- **Forecast Data**: 5-day/3-hour forecasts are available on the free tier.
- **Historical Data**: Access to over 47 years of past weather data.

4. Implementation Example (Python)

➢ You can retrieve and parse weather data using the Python requests library:

```
import requests

api_key = "YOUR_API_KEY"
city = "New York"
url =
f"https://openweathermap.org{city}&appid={api_key}&units=metric"
```

```
response = requests.get(url)
if response.status_code == 200:
    data = response.json()
    temp = data['main']['temp']
    desc = data['weather'][0]['description']
    print(f"Temperature in {city}: {temp}°C, {desc}")
else:
    print("Error fetching data")
```

(15) Google Maps Geocoding API

Q.1) Using Google Maps Geocoding API to convert addresses into coordinates.

- Using the Google Maps Geocoding API within a REST framework allows you to convert physical addresses into geographic coordinates (latitude and longitude) through automated HTTP requests.

• Key Concepts

- **Forward Geocoding**: The specific process of converting a street address (e.g., "1600 Amphitheatre Parkway, Mountain View, CA") into coordinates.
- **API Key**: A required unique identifier obtained from the Google Cloud Console that tracks usage and manages billing.
- **Response Format**: The API typically returns data in JSON or XML formats, containing detailed location information including the latitude, longitude, and formatted address.

• Implementation Steps

- **Obtain API Key**: Create a project in the Google Cloud Console, enable the Geocoding API, and generate an API key.
- **Construct Request**: Send an HTTP GET request to the following endpoint:
https://googleapis.com.

- **Parse Response**: Extract the lat and lng values from the geometry.location field in the JSON response.

Development Tools

- **Python Client Library**: Use the official Google Maps Python Client (pip install googlemaps) to simplify authentication and request handling.
- **Standard Requests**: Alternatively, use the Python Requests library to manually call the REST endpoint.

(16) GitHub API Integration

Q.1) Introduction to GitHub API and how to interact with repositories, pull requests, and issues.

- The GitHub REST API provides a powerful way to programmatically manage your development workflow. It allows you to automate repetitive tasks and integrate GitHub features directly into external applications.

• Getting Started

- **Base URL**: All API requests are made to https://api.github.com.
- **Authentication**: While public data can often be fetched without a token, most actions (like creating repositories or issues) require a Personal Access Token (PAT). You can generate one in your GitHub Developer Settings.
- **Headers**: Every request must include a User-Agent header, typically your GitHub username or application name. Authenticated requests require an Authorization: token YOUR_TOKEN header.

• Interacting with Repositories

- You can manage repository lifecycles using these primary endpoints:

- **List Repositories**: Use GET /user/repos for your own or GET /orgs/{org}/repos for an organization.

- **Create a Repository**: Send a POST request to /user/repos with a JSON body containing at least a "name".
- **Get Details**: Use GET /repos/{owner}/{repo} to retrieve specific repository information.
- **Delete a Repository**: Send a DELETE request to /repos/{owner}/{repo} (requires admin access).

• Interacting with Issues

➢ The Issues API handles tasks like assigning work and tracking bugs:

- **List Issues**: Use GET /repos/{owner}/{repo}/issues to fetch open issues for a specific repository.
- **Create an Issue**: Send a POST request to /repos/{owner}/{repo}/issues with a "title" and optional "body".
- **Update/Close an Issue**: Send a PATCH request to /repos/{owner}/{repo}/issues/{issue_number} with {"state": "closed"}.
- **Manage Comments**: Use POST /repos/{owner}/{repo}/issues/{issue_number}/comments to add a comment.

• Interacting with Pull Requests (PRs)

➢ The Pull Request API is used to manage code reviews and merges:

- **List Pull Requests**: Use GET /repos/{owner}/{repo}/pulls.
- **Create a PR**: Send a POST request to /repos/{owner}/{repo}/pulls with required fields: "title", "head" (source branch), and "base" (target branch).
- **Merge a PR**: Send a PUT request to /repos/{owner}/{repo}/pulls/{pull_number}/merge.

- **Note on Relationships**: In GitHub's API, every pull request is an issue, but not every issue is a pull request. Actions like adding labels or assignees to a PR must be done using the Issues API endpoints

(17) Twitter API Integration

Q.1) Using Twitter API to fetch and post tweets, and retrieve user data.

- Integrating the Twitter (X) API with a REST framework (such as Django REST Framework) involves obtaining credentials and using a Python library like Tweepy to handle the communication.

1. Prerequisites and Setup

- **Developer Credentials**: Register for a developer account at the X Developer Portal. Create a project and an app to generate the following:
  - API Key & API Key Secret (Consumer Keys).
  - Access Token & Access Token Secret.
  - Bearer Token.
- **Permissions**: For posting, ensure your app's user authentication settings are set to "Read and Write".
- **Library Installation**: Use pip to install the necessary library:
  pip install tweepy

2. Fetching Tweets

- You can retrieve tweets from a specific user's timeline using the API v2 Client in Tweepy.
  - **Method**: Use client.get_users_tweets(id=user_id) to fetch the most recent tweets for a specific user ID.
  - **Historical Data**: The REST API is primarily used to gather data produced before the time of collection.

3. Posting Tweets

- To programmatically post a new tweet, you must have write permissions enabled on your access tokens.
  - **Implementation**: Initialize a Tweepy Client with your four keys and use the create_tweet method:

```
import tweepy

client = tweepy.Client(consumer_key=CK,
consumer_secret=CS, access_token=AT,
access_token_secret=ATS)

client.create_tweet(text="Hello world!")
```

4. Retrieving User Data

- Retrieve profile information such as a user's display name, ID, and bio.
  - **By Username**: Use client.get_user(username='handle') to find a specific account's details.
  - **Verification**: The verify_credentials() method (in v1.1) or get_me() (in v2) can be used to return the profile data of the currently authenticated user

(18) REST Countries API Integration

Q.1) Introduction to REST Countries API and how to retrieve country-specific data.

- The REST Countries API is a free, open-source tool that provides comprehensive geographical and demographic data about countries worldwide via a RESTful interface. It is frequently used by developers to populate applications with real-world data like population, currencies, and languages.

- **Key Endpoints for Retrieving Data**

- Data is retrieved using standard HTTP GET requests.
  Replace {parameter} with your specific search term:
  - **All Countries**: Use /all to fetch the complete list.
    - *Note:* In v3.1, you may need to filter fields using the ?fields= query to avoid "bad request" errors.
    - Example: https://restcountries.com
  - **By Name**: Use /name/{name} for partial or full name matches.
    - Partial: https://restcountries.com/v3.1/name/peru
    - Full Name Match: Append ?fullText=true for exact results (e.g., /name/aruba?fullText=true).
  - **By Country Code**: Use /alpha/{code} to search by ISO 3166-1 alpha-2 or alpha-3 codes.
    - Example: https://restcountries.com/v3.1/alpha/co (Colombia).
  - **By Regional Criteria**:
    - **Region**: /region/{region} (e.g., /region/europe).

- **Capital City**: /capital/{capital} (e.g., /capital/london).
- **Currency**: /currency/{currency} (e.g., /currency/usd).
- **Language**: /lang/{language} (e.g., /lang/spanish).

• Implementation Workflow

➢ To retrieve country-specific data in a modern web application:

1. **Construct the URL**: Identify the correct endpoint based on your search criteria (e.g., name or code).
2. **Fetch Data**: Use an asynchronous function (like fetch in JavaScript with async/await) to send the request.
3. **Parse JSON**: The server returns data in JSON format, which must be parsed to access specific properties like population, flags.png, or name.common.
4. **Field Filtering**: To improve performance, append ?fields=field1,field2 to the URL to only receive the specific data your app requires.

(19) Email Sending APIs (SendGrid, Mailchimp)

Q.1) Using email sending APIs like SendGrid and Mailchimp to send transactional emails.

- Integrating transactional email APIs like SendGrid and Mailchimp into a REST framework (such as Django REST Framework) involves configuring external backends to handle high-volume, reliable delivery for account-related notifications.

1. SendGrid Integration

- SendGrid is a developer-first platform primarily used via its REST API or SMTP relay.
  - **Setup**: Create an API Key on SendGrid with "Mail Send" permissions.
  - **Django Configuration**: Use the standard django.core.mail by configuring SMTP settings or a specialized backend like django-sendgrid-v5.

```
# settings.py

EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'

EMAIL_HOST = 'smtp.sendgrid.net'

EMAIL_HOST_USER = 'apikey'  # Constant string

EMAIL_HOST_PASSWORD = 'YOUR_SENDGRID_API_KEY'

EMAIL_PORT = 587

EMAIL_USE_TLS = True
```

- **Execution**: Call send_mail() within your API views to trigger transactional messages.

2. Mailchimp Transactional Integration

➢ Mailchimp Transactional (formerly Mandrill) is a paid add-on focused on speed and high delivery rates.

- **Setup**: Generate a transactional API key and verify your sending domain with DKIM records.
- **API vs. SMTP**: You can send via the Mailchimp Transactional API using their Python SDK or configure SMTP relay for a simpler drop-in replacement.
- **DRF Integration**: Create a service class to handle the SDK calls.

```
import mailchimp_transactional as MailchimpTransactional
client =
MailchimpTransactional.Client('YOUR_MAILCHIMP_API_KEY')
message = {
    "from_email": "hello@example.com",
    "subject": "Hello world",
    "text": "Welcome to our app!",
    "to": [{"email": "recipient@example.com", "type": "to"}]
}
client.messages.send({"message": message})
```

(20) SMS Sending APIs (Twilio)

Q.1) Introduction to Twilio API for sending SMS and OTPs

➢ The Twilio REST API allows developers to programmatically send SMS and one-time passwords (OTPs) by making standard HTTPS requests.

- **Core APIs for SMS & OTP**

  - **Programmable Messaging API**: Used for sending general SMS and MMS messages. You can use it to send custom OTPs by generating a code in your application and including it in the message body.

  - **Twilio Verify API**: A managed authentication service specifically designed for OTPs. It handles code generation, delivery across multiple channels (SMS, Voice, WhatsApp, Email), and verification without requiring you to manage a separate database for codes.

- **Prerequisites for Integration**

  ➢ To use these APIs, you must first set up your environment:

  - **Twilio Account**: Sign up to receive your Account SID and Auth Token, which are used as your API credentials.

  - **Twilio Phone Number**: Purchase a Twilio number from the console to serve as the "From" sender address.

- **Helper Libraries**: Install official SDKs for languages like Python (pip install twilio) or Node.js (npm install twilio) to simplify REST requests.

• Implementation Workflow

- **Sending a Simple SMS (Python Example)**

➢ To send a message, you make a POST request to the Message resource using the Twilio Client:

python

```
from twilio.rest import Client

client = Client(ACCOUNT_SID, AUTH_TOKEN)
message = client.messages.create(
    body="Your OTP code is 123456",
    from_="+1234567890", # Your Twilio number
    to="+0987654321"     # Recipient number
)
print(message.sid)
```

• Verification with Verify API

➢ Using the Verify API, the workflow involves two steps:

- **Send OTP**: Call the Verifications endpoint with the recipient's phone number and preferred channel (e.g., sms). Twilio generates and sends the code.

- **Verify OTP**: When the user submits the code, call the VerificationChecks endpoint. Twilio returns a success or failure status.

Key Benefits

- **Global Reach**: Delivery to customers in over 180 countries.
- **Security**: Support for API keys to avoid hardcoding credentials and built-in fraud prevention tools.
- **Scalability**: Managed infrastructure handles message queuing and carrier compliance.

(21) Payment Integration (PayPal, Stripe)

Q.1) Introduction to integrating payment gateways like PayPal and Stripe.

- Integrating payment gateways like Stripe and PayPal into a Django REST Framework (DRF) application involves setting up secure API communication, handling frontend tokens, and processing webhooks to confirm transactions. The process typically uses a "headless" approach where the backend API creates a payment session, and the frontend handles the redirection or card input.

Here is an introduction to integrating both gateways.

1. General Payment Integration Workflow

1. **Setup Account**: Create merchant accounts (Stripe Dashboard / PayPal Developer Dashboard) and obtain API keys.
2. **Environment Setup**: Store secrets securely using .env files (e.g., STRIPE_SECRET_KEY).
3. **Frontend Request**: The client (React/Vue/Mobile) calls a DRF APIView to initiate a purchase.
4. **Backend Session Creation**: DRF backend creates a payment session (Checkout Session or Payment Intent) with the provider.
5. **Secure Payment**: The provider handles card data (PCI compliance) and returns a success/cancel status.
6. **Webhook Verification**: The provider sends a webhook to your DRF API to securely confirm the payment.

2. Integrating Stripe with DRF

➢ Stripe is popular for its developer-friendly API and Stripe Elements, which simplify PCI compliance by capturing card details securely.

- **Key Steps:**

- **Install SDK**: pip install stripe.
- **Create Checkout View**: Use stripe.checkout.Session.create() to generate a checkout URL, which is returned to the frontend.
- **Handle Webhooks**: Create an endpoint that receives POST requests from Stripe to verify checkout.session.completed events, ensuring payment is actually processed before updating database records.
- **Test Mode**: Use Stripe's test card numbers (e.g., 4242...) to simulate transactions.

3. Integrating PayPal with DRF

➢ PayPal provides a comprehensive REST API to create orders and authorize payments.

- **Key Steps**:

- **Install SDK/Package**: Use paypal-checkout-sdk or the django-paypal library.
- **Get Access Token**: Use client ID and secret to retrieve a Bearer token.
- **Create Order API**: Create a DRF view that calls PayPal's /v2/checkout/orders to initiate a payment session.
- **Capture Payment**: After the user approves the payment on the PayPal site, the frontend sends the order ID to your backend, which uses actions.order.capture() to finalize the payment.

(22) Google Maps API Integration

Q.1) Using Google Maps API to display maps and calculate distances between locations.

- Integrating Google Maps API with Django REST Framework (DRF) allows you to build powerful geo-spatial applications that display interactive maps and calculate precise travel metrics.

1. Displaying Maps

- To display a map, you typically use the Maps JavaScript API on the frontend (e.g., React, Vue, or a Django Template) while fetching location data from your DRF backend.
  - **Backend role**: Your DRF API should serve location coordinates (latitude and longitude) through a serializer.
  - **Frontend role**: Use the coordinates to initialize a map and place markers.
  - **Action**: You can manage markers and API keys through the Google Cloud Credentials page.

2. Calculating Distances

- For distance calculations, you can use the Distance Matrix API or the Routes API.
  - **Server-Side (Recommended)**: Use the Google Maps Python Client within your DRF views to calculate distances before returning the response.

```
import googlemaps
gmaps = googlemaps.Client(key='YOUR_API_KEY')
result = gmaps.distance_matrix('Origin City', 'Destination City')
distance = result['rows'][0]['elements'][0]['distance']['text']
```

- **Client-Side**: If you only need to show distance visually without saving it, use the Distance Matrix Service in JavaScript.

3. Integrating with DRF Serializers

➢ To include a calculated distance in your API response, use a SerializerMethodField:

- **Dynamic Calculation**: Calculate the distance based on the user's current location passed as a query parameter.
- **Field Inclusion**: Add a distance field to your serializer to return the data in your JSON response.